WEEK 2 – DATA 56000

Spring 2026

Pol Monné Parera (L30170743)
pmonneparera@lewisu.edu

1. Technical Description

1.1 Radial Basis Function Neural Network

A Radial Basis Function Neural Network (RBFNN) is a three-layer feed-forward network in which the hidden layer applies a radial (distance-based) activation function rather than the sigmoid or ReLU activations used in standard MLPs. The network maps input features to class labels via two sequential transformations: a non-linear, distance-based projection into a high-dimensional feature space, followed by a linear output mapping.

The architecture used in this assignment consists of three layers:

- Input layer: 3 neurons (R, G, B pixel values, normalized to [0, 1])
- Hidden layer: 30 neurons, each measuring how close a pixel's color is to one of 30 learned reference colors
- Output layer: 3 linear neurons (one per class), mapped to class probabilities via SoftMax

1.2 Hybrid Learning Method

In this assignment, the RBFNN has been trained using a hybrid method rather than a stochastic gradient method. With this approach the network is trained in two distinct separate steps:

1. Unsupervised (K Means): The k-means clustering algorithm is run on the training data (pixels) to find k representative colors.
2. Supervised (least squares): With the centers fixed, the output weights are computed in one shot using a mathematical formula rather than iterative training.

1.3 Pixel Classification Task

Each pixel in the image is treated as an independent sample with 3 features (R, G, B). The network learns to assign each pixel to one of three semantic color classes: Light/Background, Midtones, or Dark/Shadow.

The formulation used to assign pixels to each one of the three classes first starts by obtaining the luminescence via ITU-R BT.601 (a standard for converting RGB to luminance). It then assigns each class with the following thresholds which I have chosen arbitrarily by dividing the ranges in three equal ranges:

Class	Name	Luminance range
0	Light / Background	$Y \geq 170$
1	Midtones	$85 \leq Y < 170$
2	Dark / Shadow	$Y < 85$

2. Design of the Algorithms

2.1 Overall Pipeline

The pipeline implemented in the notebook can be observed in the steps below, with each step representing an independent stage of the code execution.

1. Load image (data.coffee(), resized to 128×128), normalize pixels to [0, 1]
2. Assign per-pixel class labels via luminance thresholds
3. Split into 80% train / 10% validation / 10% test
4. Phase 1: fit K-Means with K = 30 on training inputs to obtain centers and widths
5. Phase 2: compute RBF design matrix and solve W via regularized least squares
6. Evaluate on held-out test set, compute confusion matrix and accuracy
7. Generate full-image classification map and test generalization on unseen images

2.2 RBFNN Hybrid Learning Pseudocode

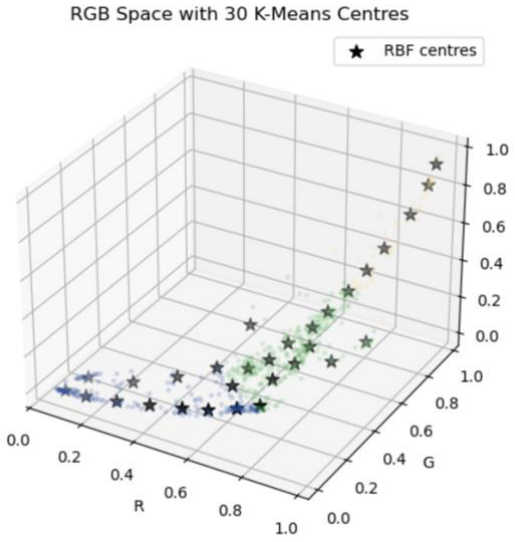
Below you will find a schema of the algorithm implemented designed in a pseudocode style:

```
// Phase 1: Find K reference colors using K-Means
centres ← KMeans(X_train, K=30)
sigmas  ← average distance between neighbouring centres

// Phase 2: For each pixel, compute similarity to each centre
FOR each pixel x_i, FOR each centre c_j:
        Phi[i,j] ← gaussian_similarity(x_i, c_j)

// Phase 3: Learn output weights (one-shot solve)
T ← one_hot(y_train)
W ← least_squares(Phi, T)

// Predict
FOR each new pixel x:
        similarities ← gaussian_similarity(x, centres)
        y_hat        ← argmax( similarities * W )
```

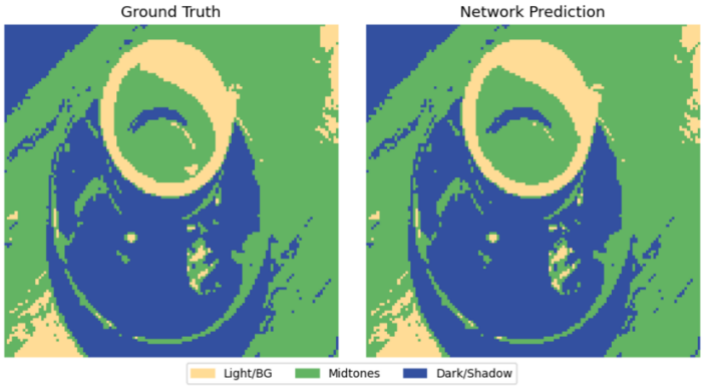
3. Results of the Algorithm

3.1 Original Image

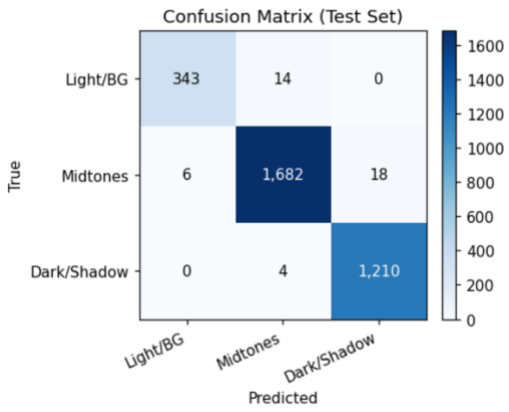
3.2 RBF Centre Distribution



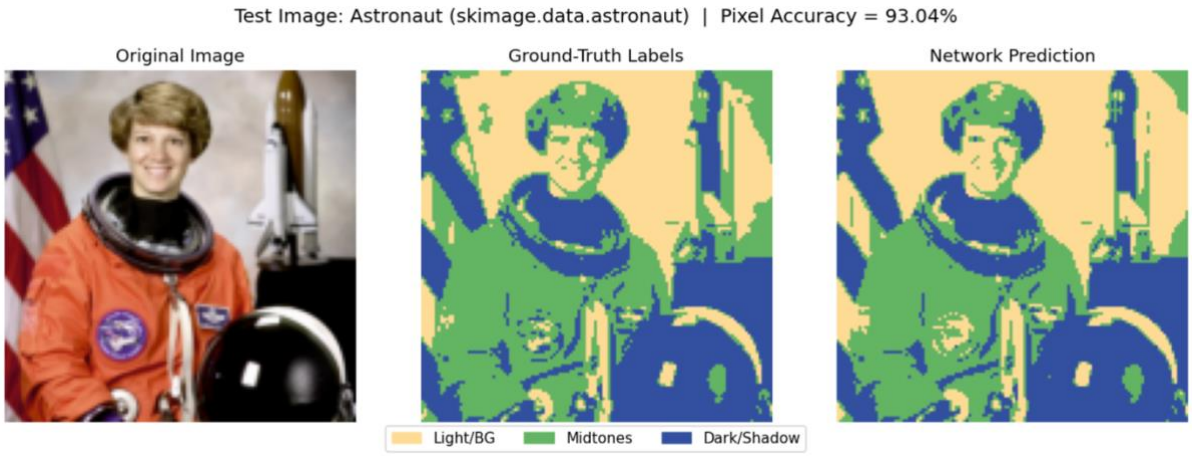
3.3 Classification Map

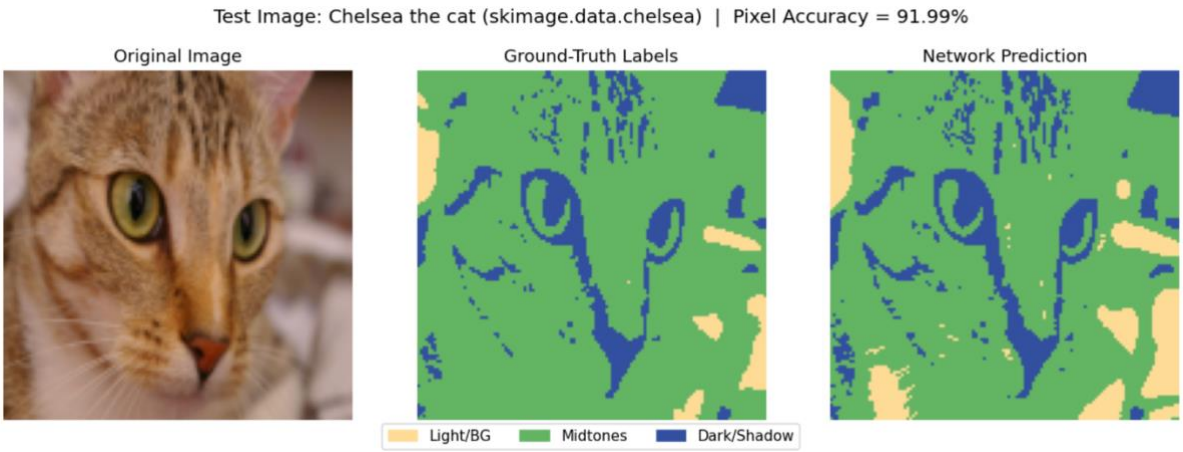


3.4 Confusion Matrix



3.5 Generalization





4. Analysis of the results

4.1 Did I obtain what I expected?

Given that class boundaries are defined by smooth luminance thresholds in RGB space, and RBF networks are universal approximators of such smooth functions, high accuracy was anticipated. It was also expected that, even though the backpropagation NN performed great, the RBFNN could outperform it in some images.

It was also expected that we would observe much faster training times with this approach that what we did with the backpropagation NN, as with it we are not training over a 120 epochs. Considering that in the hybrid approach training completes in a single pass this behavior was anticipated.

4.2 RBFNN vs BPNN Comparison

Metric	BPNN (Assignment 1)	RBFNN (Assignment 2)
Training paradigm	Iterative (SGD, 120 epochs)	One-shot (K-Means + Least Squares)
Hidden neurons	12 (Sigmoid)	30 (Gaussian RBF)
Coffee image accuracy	98.72%	98.72%
Generalisation (Astronaut)	91.94%	93.04%
Generalization (Chelsea)	92.11%	91.99%

4.3 Surprises

The main surprise was the speed at which the model completed training. Even though it was expected and theorized that this model would be more efficient than the backpropagation model implemented in week 1, it was a surprise to actually see it perform so fast.

It was also a surprise to see how much the model would underfit or overfit depending on the value of K assigned. At the end the value of K = 30 was selected as a balance point for the problem.

4.4 Conclusions

Even though the RBFNN and the BPNN are two fundamentally different philosophies for learning non-linear classifiers, both architectures perform exceptionally well with the image classification task at hands. The key characteristic to consider is the fact that the RBFNN hybrid method is deterministic and gradient-free, making it well suited for environments with training time is crucial to be optimized.

Future improvements could include adaptive sigma selection per cluster, cross-validated K selection, or extending the classification to a finer set of color classes.

5. Program Listings

```
X_tr, X_te, y_tr, y_te = train_test_split(
    X_all, y_all, test_size=0.20, random_state=42, stratify=y_all)
X_tr, X_val, y_tr, y_val = train_test_split(
    X_tr, y_tr,   test_size=0.10, random_state=42, stratify=y_tr)

print(f'Train : {len(X_tr):,}')
print(f'Val   : {len(X_val):,}')
print(f'Test  : {len(X_te):,}')
```

Figure 1 Data Splitting

```
# — Phase 1: K-Means centre placement ————————————————————————
def _fit_centers(self, X):
    km = KMeans(n_clusters=self.num_centers, n_init=10,
                max_iter=300, random_state=self.seed)
    km.fit(X)
    self.centers = km.cluster_centers_

    # Width heuristic: nearest-neighbour distance between centres
    pairwise = np.sqrt(((self.centers[:, None, :] - self.centers[None, :, :]) ** 2).sum(axis=2))
    pairwise[pairwise == 0] = np.nan
    sigma = np.nanmean(np.nanmin(pairwise, axis=1)) * self.sigma_mult
    sigma = max(float(sigma), 1e-2)
    self.sigmas = np.ones(self.num_centers) * sigma
```

Figure 2 RBFNN Hybrid - Phase 1

```python
def _fit_weights(self, Phi, T):
    # Solve W = (Phi^T @ Phi + reg*I)^-1 @ Phi^T @ T
    A = Phi.T @ Phi + self.reg * np.eye(self.num_centers)
    B = Phi.T @ T
    self.W_out = np.linalg.solve(A, B)   # (K, n_classes)
```

Figure 3 RBFNN Hybrid - Phase 2

```python
NUM_CENTERS = 30      # number of RBF centres (hidden neurons)
SIGMA_MULT = 1.0      # multiplier for the centre width heuristic
REG = 1e-4            # regularisation coefficient for least-squares

rbf = RBFNetworkHybrid(
    num_centers = NUM_CENTERS,
    sigma_mult = SIGMA_MULT,
    reg = REG,
    seed = 42,
)
print('Network configured.')
```

Figure 4 Hyperparameter Configuration

Resources

Image used: scikit-image built-in dataset — data.coffee(), a 400×600×3 color photograph of a coffee cup. Available at: https://scikit-image.org/docs/stable/api/skimage.data.html#skimage.data.coffee.

Image used: scikit-image built-in dataset — data.astronaut(), a 512×512×3 color photograph of astronaut Eileen Collins. Available at: https://scikit-image.org/docs/stable/api/skimage.data.html#skimage.data.astronaut

Image used: scikit-image built-in dataset — data.chelsea(), a 300×451×3 color photograph of a cat. Available at: https://scikit-image.org/docs/stable/api/skimage.data.html#skimage.data.chelsea